

GEOPOLITICAL THREAT PREDICTOR

Mathematical Modeling & Historical Pattern Analysis

VITyarthi Project Report

Project	Geopolitical Threat Predictor
Author	Prateek Kumar Upadhyay
Institution	VIT Bhopal University
Programme	B.Tech — Computer Science and Engineering
Application	Python terminal-based computational model
Repository	github.com/researcher-prateek-09/Geopolitical-Threat-Predictor

Prepared from direct inspection of the public GitHub repository and source code, aligned to the VITyarthi 15-section report structure.

1. Introduction

Geopolitical Threat Predictor is a Python terminal application that converts three user-supplied indicators into a modeled threat index and supplements that score with similarity analysis against historical crises. The repository describes it as an educational computational model rather than a statistically validated forecasting system.

The three indicators are inflation level, border tension level, and resource scarcity level. Each is entered from 1 to 10. The program computes a weighted current-risk score, compares the current vector with historical cases using weighted Euclidean distance, selects the five most similar cases, calculates a similarity-weighted historical-risk score, and combines the two components.

The repository explicitly states that the model does not use machine learning, regression, Bayesian probability, statistical forecasting, or large-scale datasets. Its primary focus is therefore mathematical modeling and Python programming.

1.1 Objectives

- Represent selected geopolitical conditions numerically.
- Apply weighted mathematical scoring to current conditions.
- Compare current conditions with historical crisis patterns using distance-based similarity.
- Use the most similar historical cases as contextual input.
- Produce a repeatable terminal report with intermediate scores and classification.
- Demonstrate input validation, loops, dictionaries/lists, functions, sorting, and mathematical computation.

2. Problem Statement

Geopolitical situations involve many interacting economic, political, diplomatic, social, and strategic factors. For an educational computational model, representing such a situation as a single numerical value is challenging because different indicators do not necessarily have equal influence.

The project addresses this by defining three simplified indicators with explicit weights, then adding historical context by measuring how close the current indicator vector is to predefined historical cases. The resulting index demonstrates how multiple numerical signals can be combined into a structured computational assessment.

The project is intentionally scoped down. Historical indicator values, weights, thresholds, and outcome scores are model assumptions and are not official historical measurements or probabilities.

2.1 Scope

- Terminal interaction only
- Three current indicators
- 13 historical cases embedded in source code
- Weighted current-risk calculation
- Weighted Euclidean similarity
- Top-five historical-case selection
- Similarity-weighted historical-risk calculation
- 60/40 current-vs-historical combination
- Three-category modeled classification
- Repeated analysis within a while loop

3. Functional Requirements

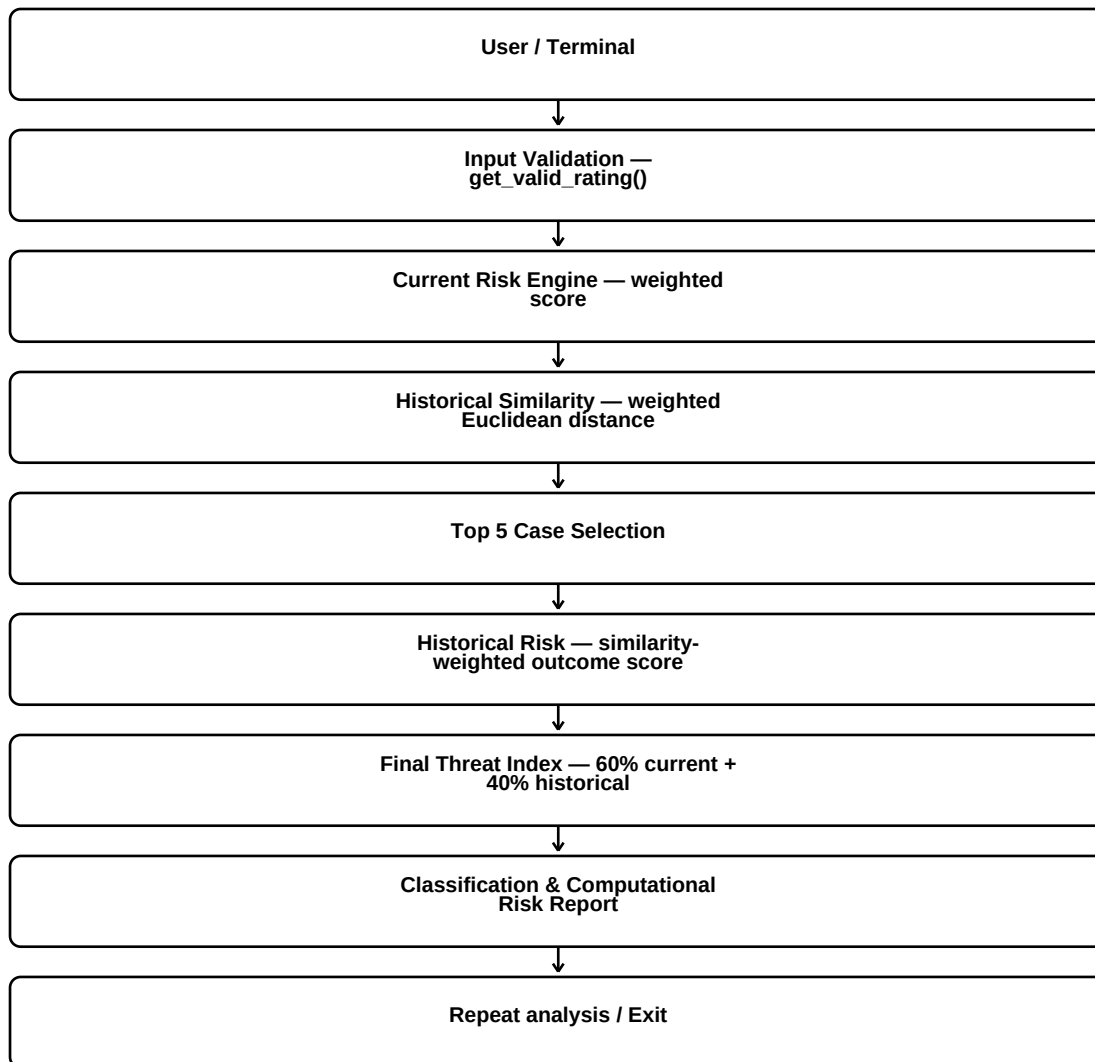
ID	Requirement
FR-01	Accept current inflation, border tension, and resource scarcity ratings.
FR-02	Validate numeric input and restrict values to 1–10.
FR-03	Allow 'bye' or 'exit' to terminate the program.
FR-04	Calculate current risk using weights 0.20, 0.30, and 0.50.
FR-05	Calculate similarity against every historical case.
FR-06	Sort historical cases and select the top five.
FR-07	Calculate similarity-weighted historical risk from outcome scores.
FR-08	Combine current and historical risk using 60% and 40%.
FR-09	Classify the final score as low, moderate, or high modeled threat.
FR-10	Print the computational report and begin another analysis cycle.

4. Non-functional Requirements

Requirement	Implementation
Usability	Clear prompts, accepted range, and structured report output.
Reliability	Rejects non-numeric, non-finite, and out-of-range input.
Maintainability	Important assumptions are exposed as named constants.
Resource Efficiency	Uses Python and the built-in math module only.
Portability	Designed for Python 3.x terminal environments.
Transparency	README documents that the result is a modeled index, not a validated probability.
Repeatability	Same inputs and embedded data produce the same output.

5. System Architecture

The current repository uses a compact single-file architecture. The primary tracked project files are README.md and geopolitical-threat-predictor.py. The Python source contains the constants, historical dataset, validation function, mathematical functions, main loop, classification, and report generation.



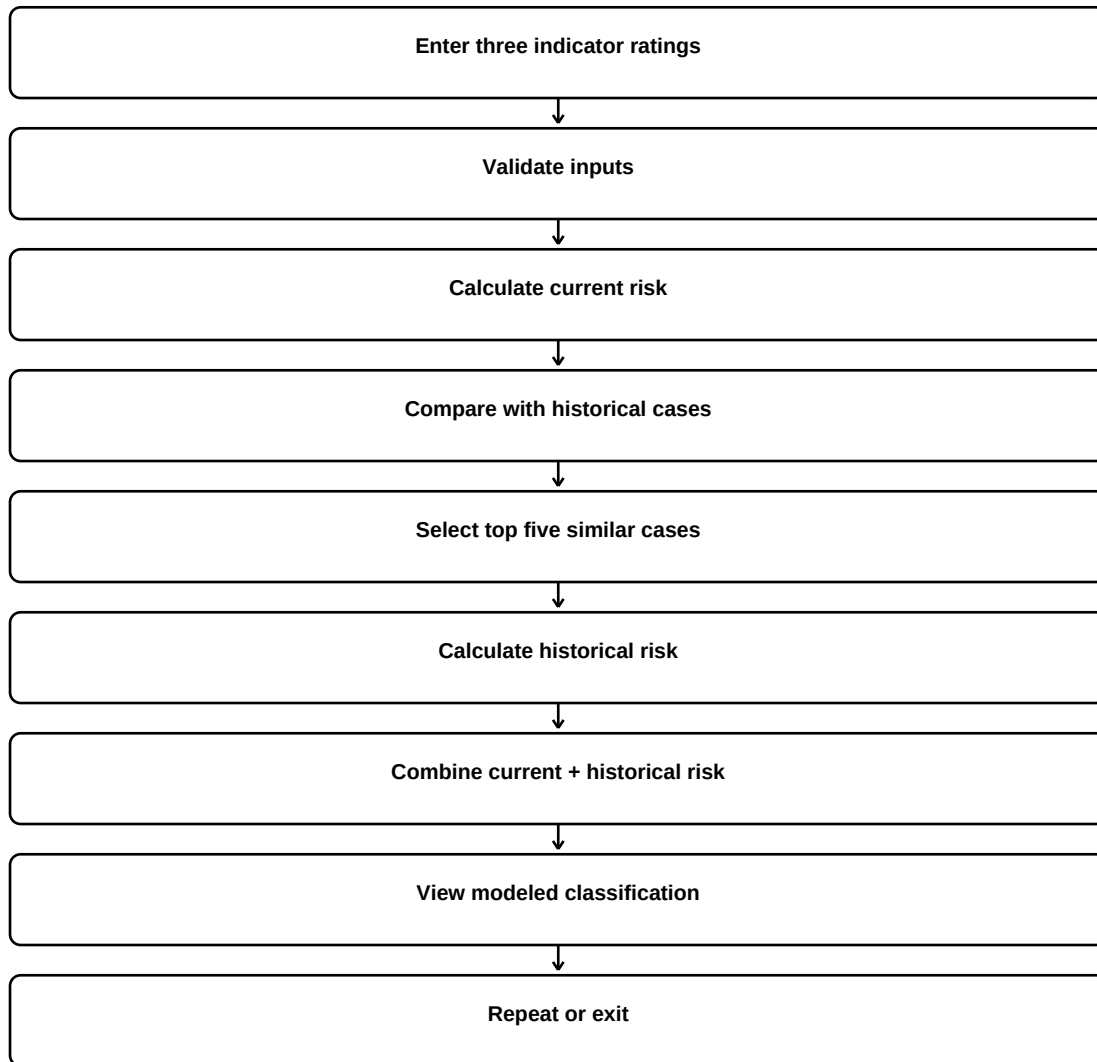
5.1 Repository Structure

```
Geopolitical-Threat-Predictor/  
■■■ README.md  
■■■ geopolitical-threat-predictor.py
```

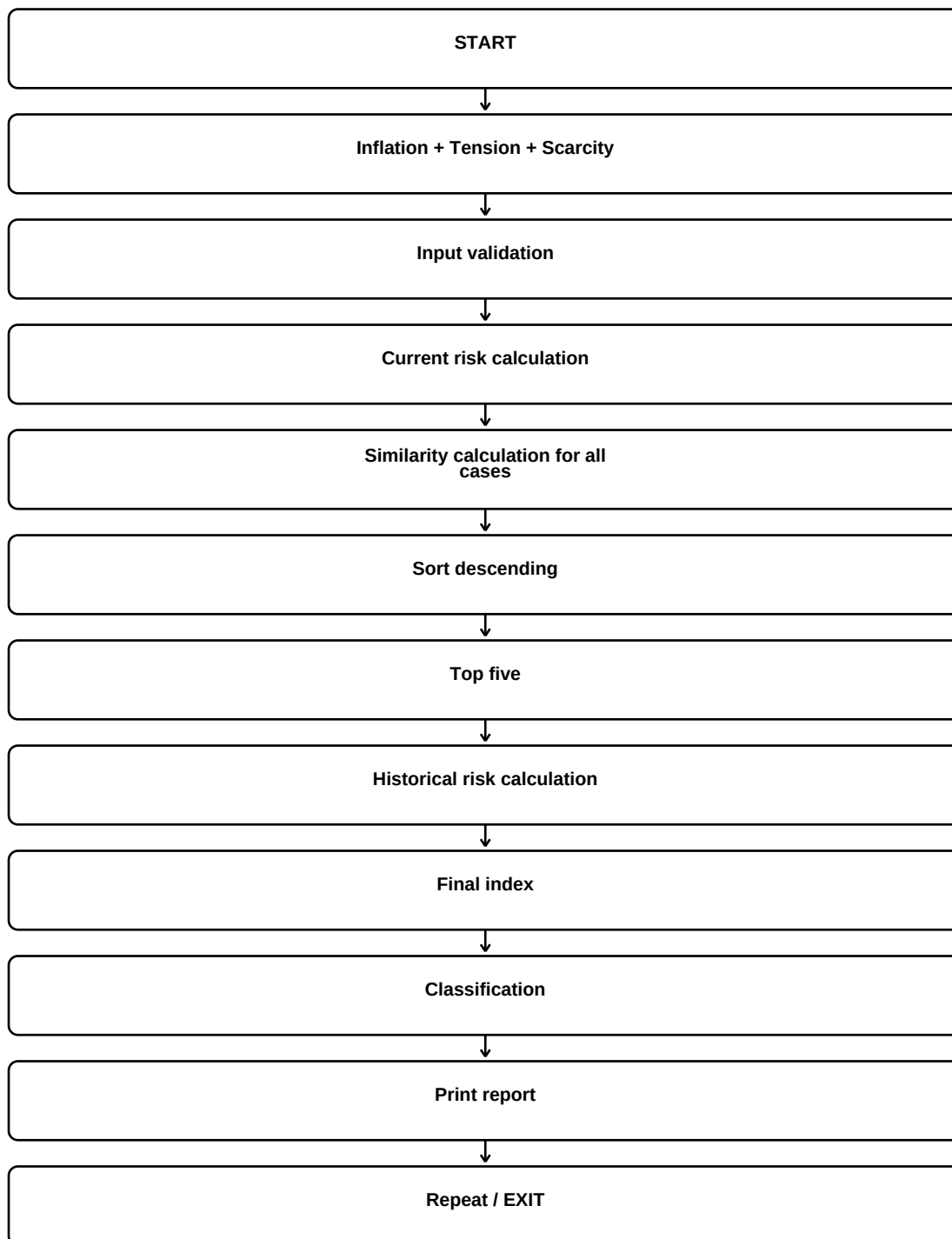
VITyarthi alignment note: the supplied guidelines recommend 5–10 meaningful modules/classes/files for coding projects. The current public repository is a compact two-file implementation, so this is a structural compliance gap that should be considered before submission.

6. Design Diagrams

6.1 Use Case / Workflow View



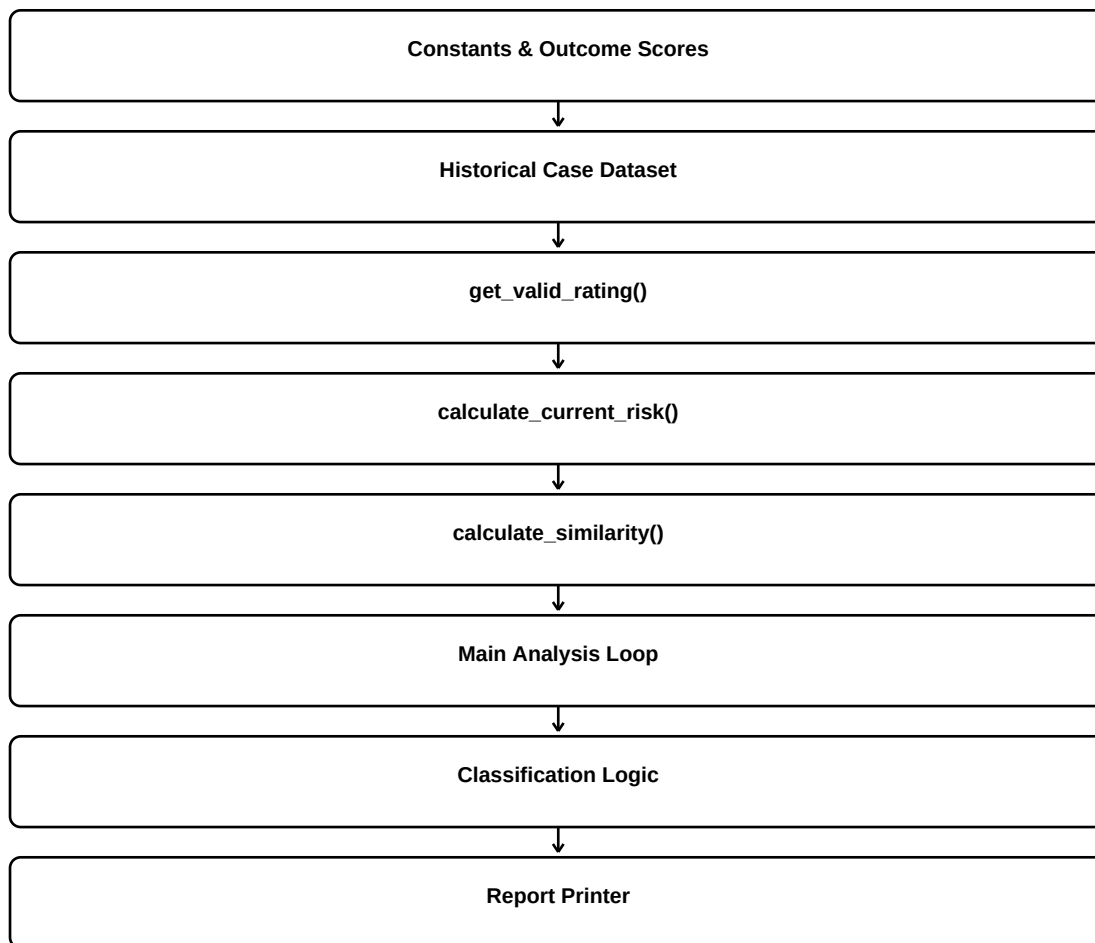
6.2 End-to-end Workflow



7. UML / Component Design

7.1 Component View

The implementation is procedural rather than class-oriented, so a component/function view is more accurate than inventing a class hierarchy.



7.2 Sequence — One Analysis

User	Program	Risk / Similarity Engine
Enter three ratings	Validate inputs	—
—	Call current-risk function	Return current risk
—	Build current-condition dictionary	Compare against all historical cases
—	Sort and select top five	Return similarity values
—	Compute historical risk	Use outcome scores × similarity
—	Combine and classify	—
—	Print report	—

7.3 ER Diagram

Not applicable. The current version has no external database or persistent storage; historical cases are stored as in-memory dictionaries.

8. Design Decisions & Rationale

Decision	Rationale
Three indicators	Keeps the model understandable for an introductory mathematical-programming project.
20/30/50 weights	Makes the relative contribution of each factor explicit.
Weighted Euclidean distance	Provides a simple measure of closeness between multi-dimensional vectors.
Top five cases	Focuses historical influence on the most similar examples.
Outcome score mapping	Allows qualitative outcomes to participate in a numerical weighted average.
60/40 combination	Makes current conditions dominant while retaining historical context.
Named constants	Makes assumptions easy to inspect and modify.
Terminal interface	Avoids GUI dependencies and keeps focus on Python logic.

9. Implementation Details

9.1 Model Parameters

Constant	Value
WEIGHT_INFLATION	0.20
WEIGHT_TENSION	0.30
WEIGHT_SCARCITY	0.50
CURRENT_RISK_WEIGHT	0.60
HISTORICAL_RISK_WEIGHT	0.40
TOP_HISTORICAL_CASES	5
HIGH_RISK_THRESHOLD	75.00
MODERATE_RISK_THRESHOLD	45.00
MIN/MAX INPUT	1.00 / 10.00

9.2 Current Risk

Current Risk = (Inflation×0.20 + Border Tension×0.30 + Resource Scarcity×0.50)×10

9.3 Similarity

Distance = $\sqrt{[0.20(I_c - I_h)^2 + 0.30(T_c - T_h)^2 + 0.50(S_c - S_h)^2]}$

The source normalizes distance against the maximum possible distance for the 1–10 range and converts it into a 0–100 similarity percentage.

9.4 Historical Risk

Historical Risk = $\Sigma(\text{Outcome Score} \times \text{Similarity}) / \Sigma(\text{Similarity})$, using the five selected cases.

9.5 Final Threat Index

Final Threat Index = (Current Risk×0.60) + (Historical Risk×0.40)

9.6 Classification

<45 → LOW MODELED THREAT

45–74.99 → MODERATE MODELED THREAT

≥75 → HIGH MODELED THREAT

10. Screenshots / Results

The application is terminal-based. The repository README gives an example with Inflation = 6, Border Tension = 8, Resource Scarcity = 7. Applying the source code's actual formulas to those values gives the following reproducible example.

10.1 Example Calculation

Current Risk = $(6 \times 0.20 + 8 \times 0.30 + 7 \times 0.50) \times 10 = 71.00$

The five closest cases are Arab Spring (~90.70%), Iranian Revolution (~90.06%), Kosovo Crisis (~88.89%), Yom Kippur War (~86.85%), and Gulf War (~86.85%). The similarity-weighted historical risk is approximately 85.92 and the combined index is approximately 76.97.

10.2 Representative Terminal Report

```
=====
COMPUTATIONAL RISK REPORT
=====
Current Inflation Level : 6.00
Border Tension Level : 8.00
Resource Scarcity Level: 7.00
-----
Current Risk Score : 71.00 / 100.00
Historical Risk Score : 85.92 / 100.00
Calculated Threat Index: 76.97 / 100.00
-----
TOP 5 SIMILAR HISTORICAL CASES
Arab Spring - ~90.70%
Iranian Revolution - ~90.06%
Kosovo Crisis - ~88.89%
Yom Kippur War - ~86.85%
Gulf War - ~86.85%
=====
```

This is a modeled output generated from the project's assumptions; it is not an independently validated forecast or probability.

11. Testing Approach

The public repository does not contain a dedicated automated test suite. Testing for the current version should therefore focus on manual/functional verification of input validation, mathematical calculations, historical ranking, classification boundaries, repeated execution, and exit behavior.

Test	Procedure	Expected
Valid inputs	Three values 1–10	Report is generated
Text input	Enter abc	Rejected and retried
Non-finite input	Enter nan/inf	Rejected using <code>math.isfinite()</code>
Out-of-range	Enter 0 or 11	Rejected and retried
Exit	Enter bye/exit	Program terminates
Repeated analysis	Finish one report	Next analysis starts
Similarity order	Use known input vector	Cases sorted descending
Thresholds	Test scores around 45 and 75	Correct category boundary

11.1 Testing Limitation

No unittest/pytest suite is present in the public repository, so this report does not claim a numerical automated-test pass rate. Adding unit tests for `get_valid_rating()`, `calculate_current_risk()`, `calculate_similarity()`, historical aggregation, and classification would strengthen the project.

12. Challenges Faced

- Reducing a complex geopolitical problem to a small number of numerical indicators.
- Choosing explicit weights while documenting that they are model assumptions.
- Comparing current conditions with historical cases whose indicator values are themselves normalized model inputs.
- Converting distance into an interpretable similarity percentage.
- Combining qualitative historical outcomes with quantitative similarity scores.
- Designing validation for invalid numbers, non-finite values, range violations, and exit commands.
- Keeping the program reusable through a loop while retaining a clean exit.

12.1 Key Methodological Limitation

The largest limitation is model validity rather than syntax: historical indicator values, outcome scores, weights, and thresholds are assumptions embedded in the project. The output should therefore be interpreted as a deterministic modeled index produced by the defined rules.

13. Learnings & Key Takeaways

- Converted qualitative concepts into numerical variables for computational modeling.
- Applied weighted sums to combine indicators.
- Applied Euclidean distance to compare multi-dimensional vectors.
- Used normalization to convert distance into bounded similarity.
- Used sorting and slicing to identify the most similar cases.
- Used dictionaries/lists to represent structured historical data.
- Separated validation and mathematical operations into functions.
- Used named constants to expose model assumptions.
- Distinguished deterministic scoring from statistically validated prediction.
- Recognized the need for testing and empirical validation before real-world interpretation.

13.1 Academic Relevance

The project demonstrates Python fundamentals, functions, loops, conditionals, dictionaries, lists, sorting, exception handling, the math module, numerical formulas, normalization, and algorithmic decomposition.

14. Future Enhancements

- Expand the historical dataset and document data provenance.
- Replace manually assigned historical values with carefully sourced datasets.
- Add indicators such as political stability, trade dependence, energy exposure, diplomacy, and demographics.
- Add country-specific and time-series analysis.
- Perform back-testing against historical situations.
- Add statistical calibration and uncertainty estimates if sufficient data becomes available.
- Build graphical visualization/dashboard capabilities.
- Split the monolithic source into modules for data, validation, scoring, similarity, classification, reporting, and main.
- Add automated unit/integration tests.
- Add persistent storage or database support if the dataset grows.
- Automate data updates while preserving provenance and versioning.

14.1 Recommended VITyarthi Structural Enhancement

The supplied VITyarthi guidelines recommend 5–10 meaningful modules/classes/files for coding projects. A future structure could include `constants.py`, `data.py`, `validation.py`, `scoring.py`, `similarity.py`, `classification.py`, `reporting.py`, `main.py`, and `tests/`. This would improve maintainability and better satisfy the structural expectation without changing the core mathematical model.

15. References

1. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.
2. Geopolitical Threat Predictor — GitHub repository:
<https://github.com/researcher-prateek-09/Geopolitical-Threat-Predictor>
3. Geopolitical Threat Predictor README.md — overview, objectives, workflow, formulas, limitations, run instructions, technologies, and author information.
4. geopolitical-threat-predictor.py — implementation of constants, historical dataset, input validation, current-risk calculation, weighted Euclidean similarity, historical-risk aggregation, classification, and report generation.
5. Python Standard Library — math module.

Submission Readiness Notes

- 15-section report structure included.
- Actual repository architecture documented.
- Mathematical formulas and assumptions documented.
- Historical dataset and scoring system documented.
- Workflow and sequence diagrams included.
- Representative calculation/output included.
- Limitations explicitly documented.
- Dedicated automated tests are not present in the public repository.
- The current repository is a compact two-file implementation and does not yet meet the VITyarthi 5–10 module/file recommendation.